

Ant-Game 规范

每个步骤结尾的**规范**一节对本实验中的接口与数据格式给出统一说明，便于查阅与实现。

1. AI 接口规范

1.1 类与方法签名

AI 模块必须提供继承自 `BaseAgent` 的 `AI` 类，并实现以下方法：

```
def choose_bundle(  
    self,  
    state: BackendState,  
    player: int,  
    bundles: list[ActionBundle] | None = None,  
) -> ActionBundle:
```

- **state**：当前游戏状态，类型为 `SDK.backend.state.BackendState`（或实现 `BackendState` 协议的类），只读使用；不应在 `choose_bundle` 内修改。
- **player**：当前 AI 所代表的玩家，0 表示先手，1 表示后手。
- **bundles**：候选动作列表；若为 `None`，可通过 `self.list_bundles(state, player)` 获取。

1.2 返回值

类型：`ActionBundle`，即一个动作组合，包含 `operations: tuple[Operation, ...]`。

每个 `Operation` 对应协议中的一条操作，最终由 `choose_operations()` 转为协议格式发送给评测系统。

1.3 调用约定

- `choose_bundle` 在己方每回合内被调用一次；调用方会将返回的 `ActionBundle.operations` 依次执行并发送。
- 若 AI 继承 `BaseAgent`，可通过 `choose_operations()` 自动将 `ActionBundle` 转为 `list[Operation]`；评测入口 `AI/main.py` 会通过 `ProtocolSession` 完成协议通信。

- 规则、合法性判断由 `SDK.backend` 负责；非法操作会被拒绝，不会修改状态。

2. 操作编码

操作类型与参数格式如下。协议格式为一行若干整数，第一个整数为操作类型。

类型	名称	协议格式	示例
11	建造防御塔	11 x y	11 11 1 在 (11,1) 建造防御塔
12	升级防御塔	12 towerId towerTypeId	12 1 31 将 ID 为 1 的塔升级为 31 类型 (Mortar+)
13	降级/拆除防御塔	13 towerId	13 1 降级或拆除 ID 为 1 的塔
21	闪电风暴	21 x y	21 9 9 在 (9,9) 释放闪电风暴
22	EMP 轰炸	22 x y	22 9 9 在 (9,9) 释放 EMP
23	引力护盾	23 x y	23 7 8 在 (7,8) 释放引力护盾
24	紧急回避	24 x y	24 7 8 在 (7,8) 释放紧急回避
31	升级生产流水线	31	提升基地出兵速度
32	升级高级护甲	32	提升工蚁最大生命值

所有坐标均为六边形地图上的格点坐标（even-q 坐标系）；地图边长为 10，中心约在 (9,9)。防御塔建造区域、可移动区域等详见 `README.md` 与 `SDK.utils.constants`（如 `HIGHLAND_CELLS`、`PLAYER_BASES`）。

3. BackendState 与观测

3.1 BackendState 主要属性

属性	类型	含义
round_index	int	当前回合数（从 0 开始）
terminal	bool	游戏是否已结束

属性	类型	含义
winner	int None	胜者 (0/1 或 None)
towers	list[Tower]	所有防御塔
ants	list[Ant]	所有工蚁
bases	list[Base]	双方基地
coins	list[int]	两方金币 , coins[0] 为先手
weapon_cooldowns	ndarray	超级武器冷却 (按武器类型)
active_effects	list[WeaponEffect]	当前生效的超级武器效果
pheromone	ndarray	信息素 (内部使用)

3.2 常用方法

方法	含义
<code>towers_of(player)</code>	获取某方所有防御塔
<code>ants_of(player)</code>	获取某方所有工蚁
<code>tower_count(player)</code>	某方防御塔数量
<code>build_tower_cost(count)</code>	建造下一座塔的价格
<code>can_apply_operation(player, op, pending)</code>	操作是否合法
<code>apply_operation(player, op)</code>	执行操作并更新状态

3.3 Tower 主要属性

属性	含义
tower_id	塔 ID
player	归属玩家
x, y	坐标
tower_type	类型 (见 TowerType 枚举)
damage, attack_range	伤害、攻击范围

3.4 Ant 主要属性

属性	含义
ant_id	工蚁 ID
player	归属玩家
x, y	坐标
hp, level	当前生命、等级 (0/1/2)
age	年龄 (当前回合 - 生成回合)
status	状态 (存活/到达/死亡/老死/冻结)
behavior	行为类型 (默认/保守/随机/蛊惑/免控)

3.5 训练观测 (SDK)

若使用 `SDK.training` , 观测与动作空间由 `training/env.py` 定义 ; `ActionCatalog` 将 `BackendState` 转为候选 `ActionBundle` 列表 , 训练策略从中选择动作索引。

4. 规则与语义约束 (简述)

- **胜负** : 任一方基地血量 ≤ 0 则对方立即获胜 ; 若回合数达到 512 , 则按基地血量、击败蚂蚁数、超级武器使用次数、AI 用时等判定 , 详见 `README.md` 。
- **回合流程** : 双方轮流操作 ; 每方在己方回合内发出一组操作 (建造/升级/降级塔、超级武器、基地升级等) , 然后进入结算 (防御塔攻击、蚂蚁移动、信息素更新、工蚁生成等) 。
- **合法性** : 所有操作由 `BackendState.can_apply_operation()` 进行合法性检查 ; 非法操作返回 `False` 且不改变状态。
- **时间** : 每回合 AI 运行时间上限为 1 秒。

更完整的规则与架构见仓库根目录 `README.md` 。仓库与代码结构 (`SDK/` 、 `AI/` 、 `tools/` 、 `game/` 等) 见「实验要求」文档。